



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

- сообщения на inStudy

АУТЕНТИФИКАЦИЯ И АВТОРИЗАЦИЯ В NODE.JS

2026 Г.

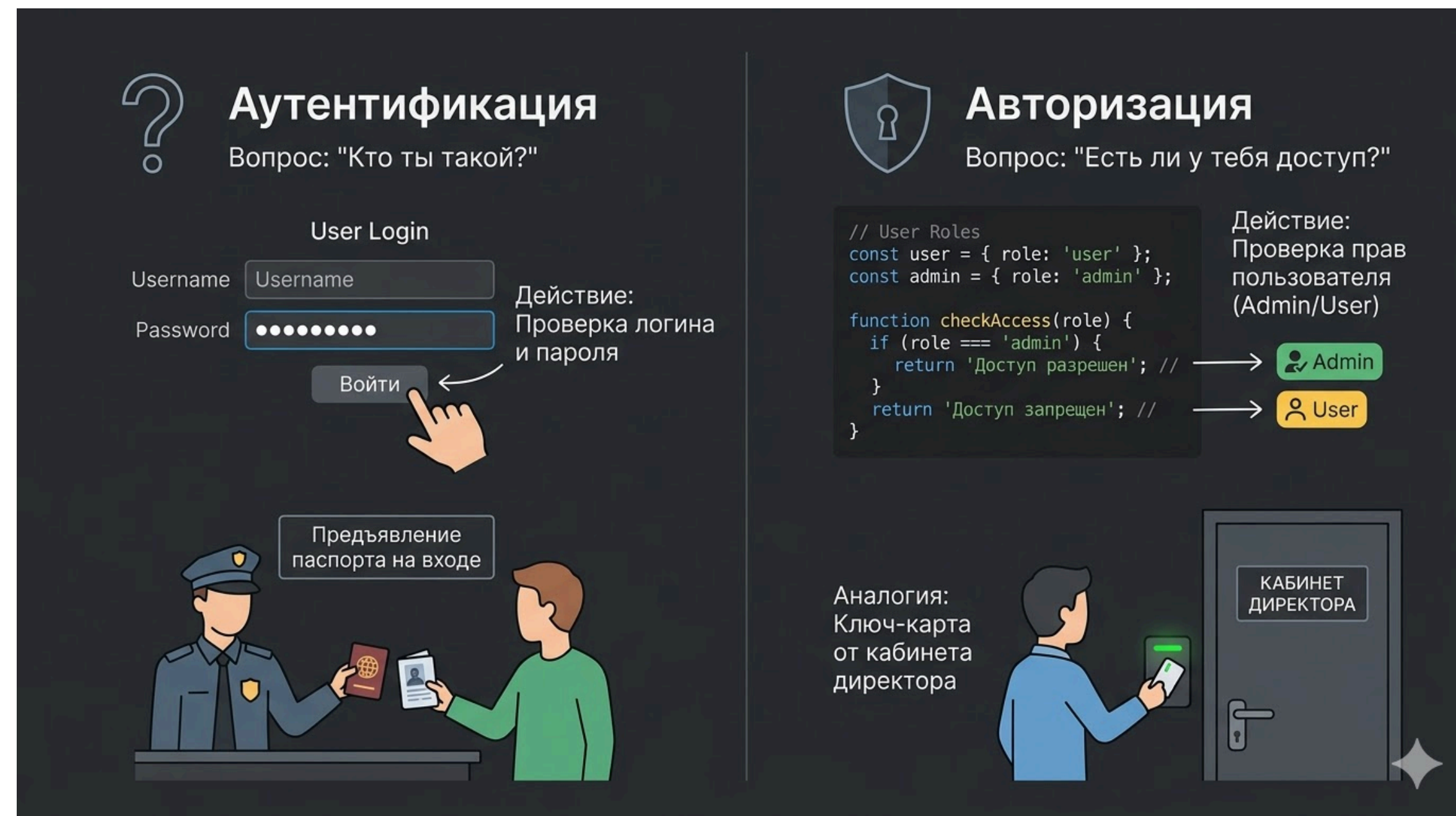


ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

В ЧЁМ РАЗНИЦА?

АУТЕНТИФИКАЦИЯ ПРОТИВ АВТОРИЗАЦИИ



1. Аутентификация

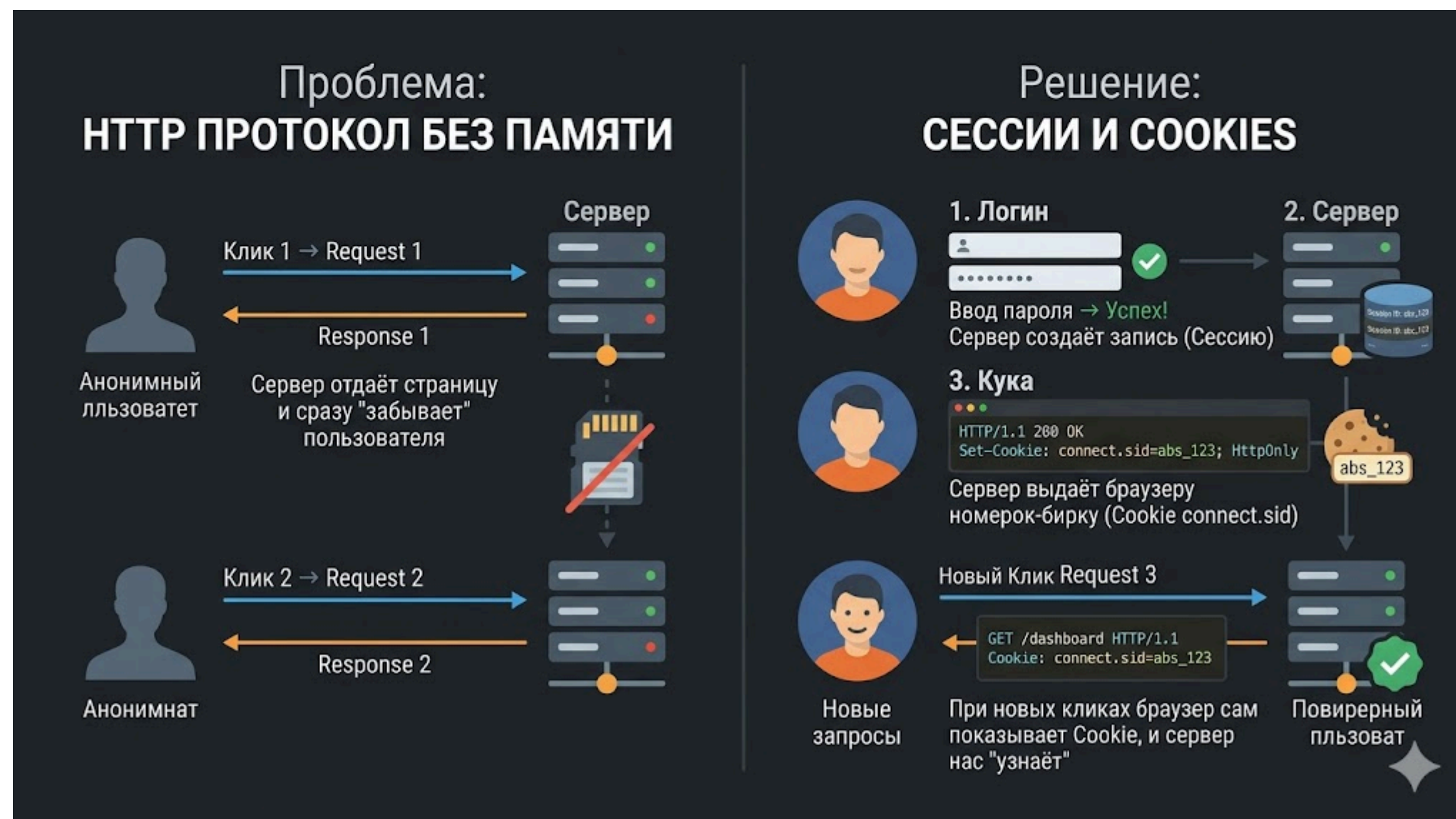
- Вопрос: "Кто ты такой?"
- Действие: Проверка логина и пароля
- Аналогия: Предъявление паспорта на входе

2. Авторизация

- Вопрос: "Есть ли у тебя доступ?"
- Действие: Проверка прав пользователя (Admin/User)
- Аналогия: Ключ-карта от кабинета директора

ПОЧЕМУ САЙТЫ НАС ЗАБЫВАЮТ?

РЕШАЕМ ПРОБЛЕМУ ЧЕРЕЗ СЕССИИ И COOKIES



1. Проблема: HTTP — протокол "без памяти"

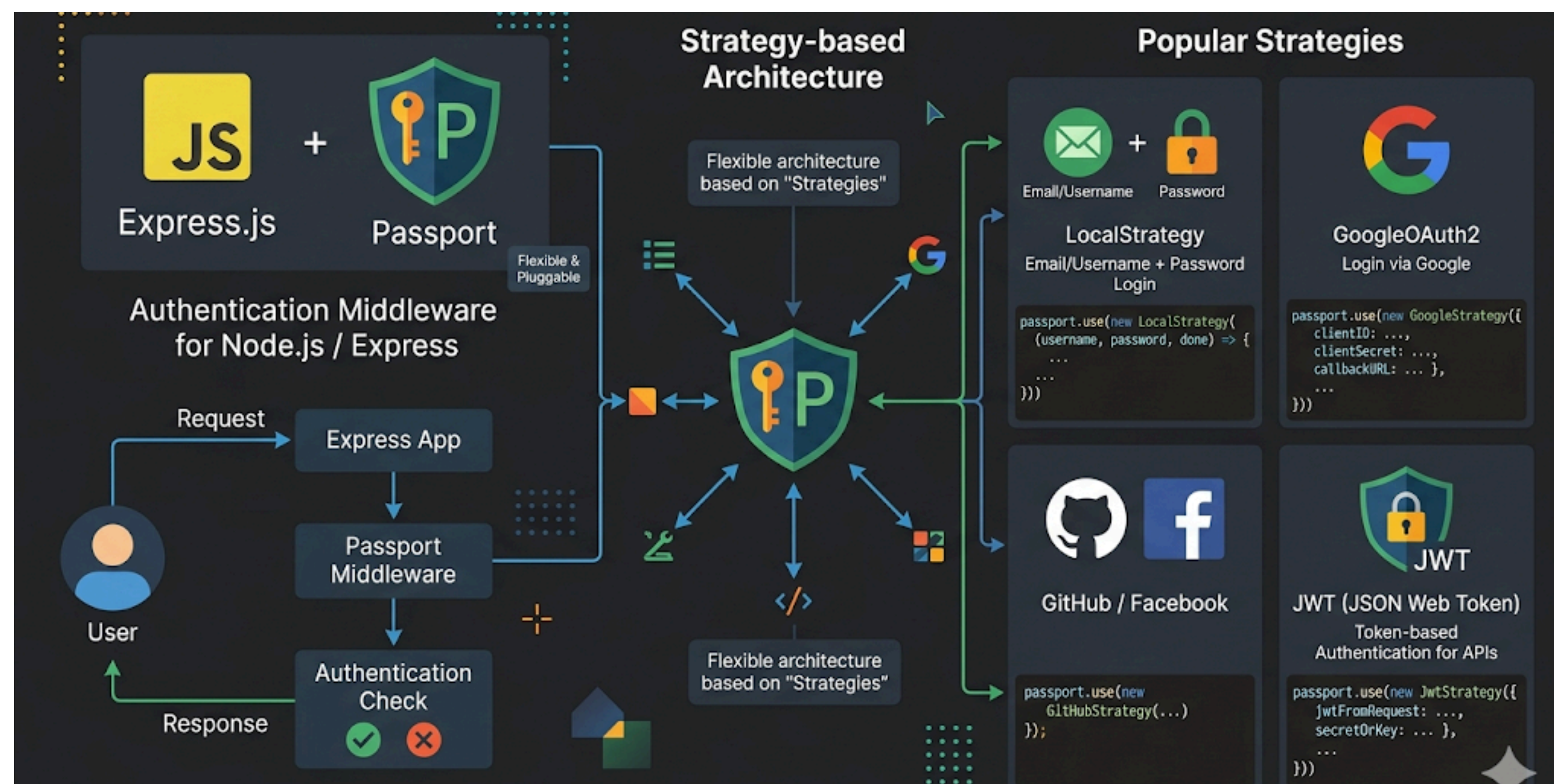
- Сервер отдаёт страницу и сразу "забывает" пользователя
- При каждом клике всё начинается с нуля

2. Решение: Сессии и Cookies

- а. Ввод пароля → Успех!
- б. Сервер создаёт запись (Сессию)
- в. Сервер выдаёт браузеру номерок-бирку (Cookie connect.sid)
- д. При новых кликах браузер сам показывает Cookie, и сервер нас "узнаёт"

PASSPORT.JS

ГЛАВНЫЙ ОХРАННИК В МИРЕ NODE.JS



1. Что такое **Passport**?

- Популярная библиотека (Middleware) для аутентификации в Node.js / Express
- Гибкая архитектура на основе "Стратегий"

2. Популярные стратегии:

- **LocalStrategy** (вход по Email/Username и Паролю)
- **GoogleOAuth2** (вход через Google)
- **GitHub** / Facebook
- **JWT** (работа с токенами для API)

ПАРОЛИ И ХЕШИРОВАНИЕ

BCRYPT

❌ Как НЕЛЬЗЯ хранить

Пароли хранятся в открытом виде

Абсолютно небезопасно при утечке БД!

Абсолютно небезопасно при утечке БД!

```
// Example of storing plain password
const user = { username: 'user1', password: '12345' };
db.save(user); // INSECURE
```

✅ Как ПРАВИЛЬНО (bcryptjs)

Хеширование — необратимое шифрование пароля

```
// Example using bcryptjs to store hash
const bcrypt = require('bcryptjs');
const password = '12345';
const saltRounds = 10;
bcrypt.hash(password, saltRounds, function(err, hash) {
  // db.save({ username: 'user1', password: hash }); // SECURE
});
```

Сервер не знает пароль, он лишь сравнивает хеш при входе

1. ❌ Как НЕЛЬЗЯ хранить пароли в базе: **password: "12345"** — Абсолютно небезопасно при утечке БД!

2. ✅ Как ПРАВИЛЬНО (bcryptjs): password: **"\$2a\$10\$vI8aWB..."**

- Хеширование — необратимое шифрование пароля
- Сервер не знает пароль, он лишь сравнивает хеш при входе

ПРИМЕНЕНИЕ СЕССИЙ И PASSPORT.JS

ПРАКТИКА.ПОДГОТАВЛИВАЕМ ПРОЕКТ

```
1  const express = require('express');
2  const session = require('express-session');
3  const passport = require('passport');
4  const LocalStrategy = require('passport-local');
5  const bcrypt = require('bcryptjs');
6
7  const app = express();
8  app.use(express.json()); // Для парсинга JSON в теле запроса
9
10 // ФЕЙКОВАЯ БАЗА ДАННЫХ (в памяти)
11 const usersDb = [];
12
```

1. Импортируем необходимые зависимости и библиотеки.

ПОДКЛЮЧЕНИЕ EXPRESS-SESSION

ПРАКТИКА. НАСТРОЙКА СЕССИЙ

```
10 // ФЕЙКОВАЯ БАЗА ДАННЫХ (в памяти)
11 const usersDb = [];
12
13 // =====
14 // 1. НАСТРОЙКА СЕССИЙ
15 // =====
16 // Сессия – это память сервера о пользователе.
17 // При первом визите сервер выдаст браузеру печеньку (cookie) с ID.
18 app.use(session({
19   secret: 'super-secret-key',    // Ключ для подписи cookie
20   resave: false,                 // Не пересохранять сессию без изменений
21   saveUninitialized: false,      // Не создавать сессию "пустым" посетителям
22   cookie: { maxAge: 1000 * 60 * 60 * 24 } // Сессия живет 24 часа
23 }));
24
```

1. Сессия – это память сервера о пользователе.
2. При первом визите сервер выдаст браузеру печеньку (cookie) с ID.

ПОДКЛЮЧЕНИЕ PASSPORT.JS

ПРАКТИКА. ИНИЦИАЛИЗАЦИЯ PASSPORT.JS

```
saveSession: false, // не создавать сессию пустым посетителем
cookie: { maxAge: 1000 * 60 * 60 * 24 } // Сессия живет 24 часа
});

// =====
// 2. ИНИЦИАЛИЗАЦИЯ PASSPORT.JS
// =====
app.use(passport.initialize()); // Подключаем Passport
app.use(passport.session()); // Связываем Passport с сессиями Express
```

1. Подключаем Passport
2. Связываем Passport с сессиями Express

СТРАТЕГИЯ АВТОРИЗАЦИИ (КАК МЫ ВХОДИМ)

ПРАКТИКА. ИНИЦИАЛИЗАЦИЯ PASSPORT.JS

```
//
passport.use(new LocalStrategy(
  {
    usernameField: 'email',    // Ищем логин в поле req.body.email
    passwordField: 'password'  // Ищем пароль в req.body.password
  },
  async (email, password, done) => {
    // ШАГ 1: Поиск пользователя в базе
    const user = usersDb.find(u => u.email === email);
    if (!user) {
      return done(null, false, { message: 'Пользователь не найден' });
    }

    // ШАГ 2: Проверяем пароль (сравниваем то, что ввели, с захешированным паролем)
    const isValid = bcrypt.compareSync(password, user.password);
    if (!isValid) {
      return done(null, false, { message: 'Неверный пароль' });
    }

    // ШАГ 3: Успешный вход! Отдаем пользователя системе
    return done(null, user);
  }
));
```

1. Подключим и настроим стратегию авторизации.
2. **LocalStrategy** принимает **2 параметра: объект с опциями** и **middleware** для верификации пользователя.
3. **Перый аргумент:** имена полей логина и пароля.
4. **Второй аргумент:** middleware — принимает параметры `username`, `passport` и `done`.
5. В **done**, вторым аргументом, передается объект пользователя.

СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ

ПРАКТИКА. ИНИЦИАЛИЗАЦИЯ PASSPORT.JS

```
// =====  
// 4. СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ  
// =====  
// Сериализация: ЧТО сохранить в cookie после входа? (обычно только email или ID)  
passport.serializeUser((user, done) => {  
  done(null, user.email);  
});  
  
// Десериализация: КАК получить пользователя при КАЖДОМ следующем клике?  
passport.deserializeUser((email, done) => {  
  const user = usersDb.find(u => u.email === email);  
  done(null, user); // Теперь пользователь будет доступен в req.user  
});
```

1. Зачем вообще это нужно

а. Когда пользователь залогинился нужно:

- запомнить, кто он
- но не хранить весь объект пользователя в сессии (это небезопасно и неэффективно)

2. Поэтому используется схема:

- а. В сессию сохраняется только ID пользователя
- б. При каждом запросе по этому ID достается полный пользователь из базы

МАРШРУТЫ (API). РЕГИСТРАЦИЯ

ПРАКТИКА. ОТ РЕГИСТРАЦИИ ДО ЗАКРЫТОГО ПРОФИЛЯ

1. Регистрируем пользователя.
2. Хешируем пароль перед сохранением!
(никогда не храните пароли текстом)

```
86 // Регистрация
87 app.post('/register', (req, res) => {
88   const { name, email, password } = req.body;
89
90   // Проверка занятости email
91   if (usersDb.find(u => u.email === email)) {
92     return res.status(400).json({ status: 'error', message: 'Email уже занят' });
93   }
94
95   // Хешируем пароль перед сохранением! (никогда не храните пароли текстом)
96   const hashedPassword = bcrypt.hashSync(password, 10);
97
98   const newUser = { id: Date.now(), name, email, password: hashedPassword };
99   usersDb.push(newUser); // Сохраняем "в базу"
100
101   res.status(201).json({ status: 'success', message: 'Пользователь зарегистрирован!' });
102 });
103
```

МАРШРУТЫ (API). ВХОД

ПРАКТИКА. ОТ РЕГИСТРАЦИИ ДО ЗАКРЫТОГО ПРОФИЛЯ

```
// Вход (использует Passport LocalStrategy)
app.post('/login', passport.authenticate('local'), (req, res) => {
  // Сюда попадем ТОЛЬКО если LocalStrategy отдала done(null, user)
  res.json({
    status: 'success',
    message: 'Успешный вход!',
    user: { name: req.user.name, email: req.user.email }
  });
});
```

1. Маршрут входа для пользователя.

2. **passport.authenticate('local')** — Самый важный этап

- Это middleware (промежуточный обработчик) от Passport.js.
- Что он делает:
 - Автоматически достает **username** и **password** из тела запроса (из req.body).
 - Вызывает **LocalStrategy** (стратегию проверки логина/пароля), которую настроили ранее в коде.
 - Внутри **LocalStrategy**: ищем пользователя в базе данных по email и сравниваем пароль (с помощью bcrypt).
- **Результат** работы authenticate:
 - **Если пароль неверный** или пользователь не найден: LocalStrategy вызывает done(null, false) или done(error). Passport автоматически вернет клиенту ошибку (например, 401 Unauthorized), и код внутри (req, res) ⇒ {...} выполнен НЕ БУДЕТ.
 - **Если проверка прошла успешно**: LocalStrategy вызывает done(null, user), передавая объект user (с информацией из базы данных).

МАРШРУТЫ (API). ЗАЩИЩЕННЫЙ МАРШРУТ ПРОФИЛЯ

ПРАКТИКА. ОТ РЕГИСТРАЦИИ ДО ЗАКРЫТОГО ПРОФИЛЯ

```
15 // Защищенный маршрут профиля
16 app.get('/profile', (req, res) => {
17     // Метод req.isAuthenticated() добавляется Passport-ом
18     if (req.isAuthenticated()) {
19         res.json({
20             status: 'success',
21             message: 'Добро пожаловать в профиль!',
22             user: { name: req.user.name, email: req.user.email }
23         });
24     } else {
25         res.status(401).json({ status: 'error', message: 'Вы не вошли в сис
26     }
27 });
```

1. Метод `req.isAuthenticated()` добавляется Passport-ом в аргумент `req`.

МАРШРУТЫ (API). ВЫХОД И ЗАПУСК

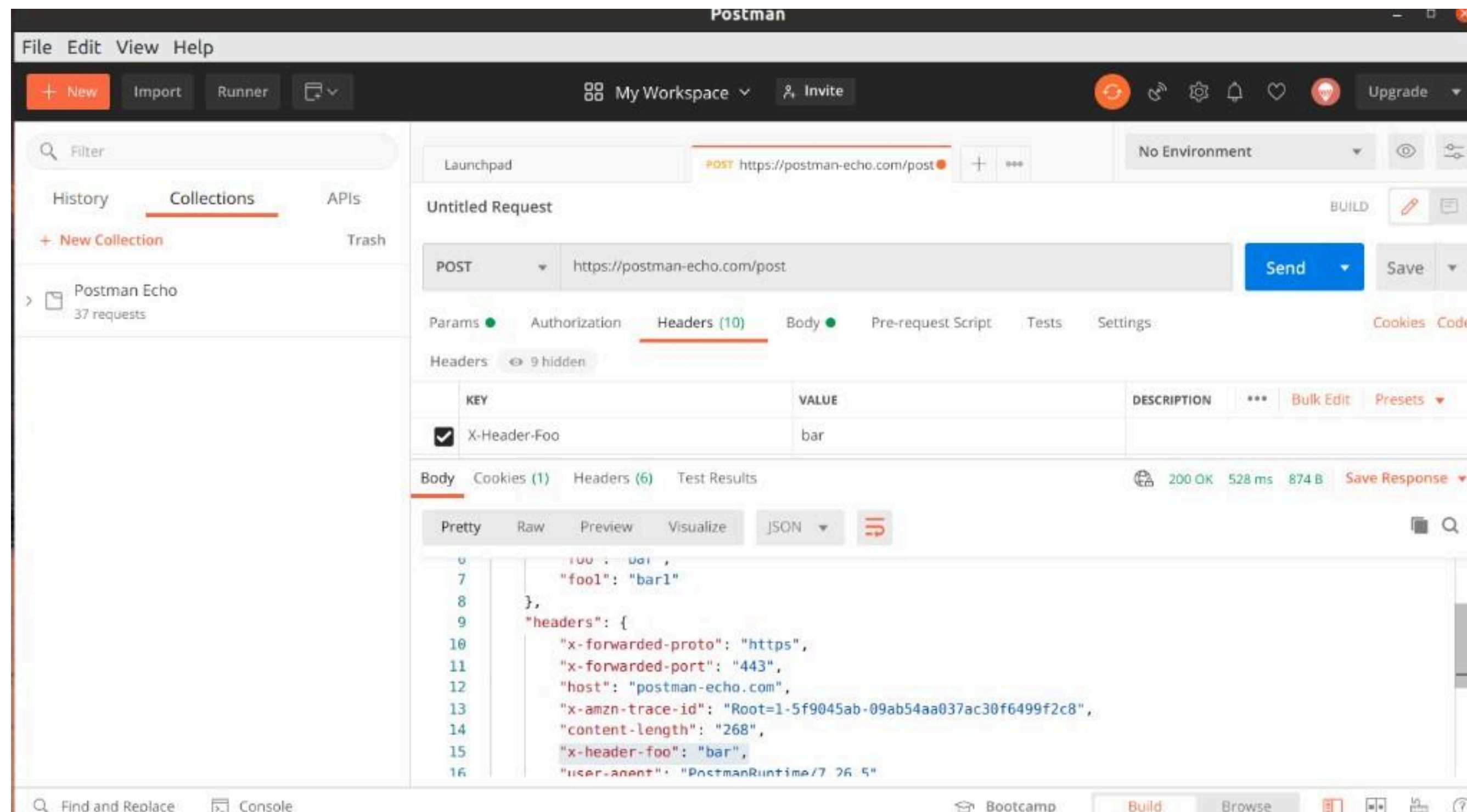
ПРАКТИКА. ОТ РЕГИСТРАЦИИ ДО ЗАКРЫТОГО ПРОФИЛЯ

```
// Выход
app.post('/logout', (req, res) => {
  req.logout((err) => {
    if (err) return res.status(500).json({ status: 'error', message: 'Ошибка при выходе' });
    res.json({ status: 'success', message: 'Вы вышли из системы' });
  });
});

// =====
// 6. ЗАПУСК
// =====
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`✅ Сервер запущен! Откройте http://localhost:${PORT}`)
});
```

POSTMAN — ЗАЧЕМ ОН НУЖЕН

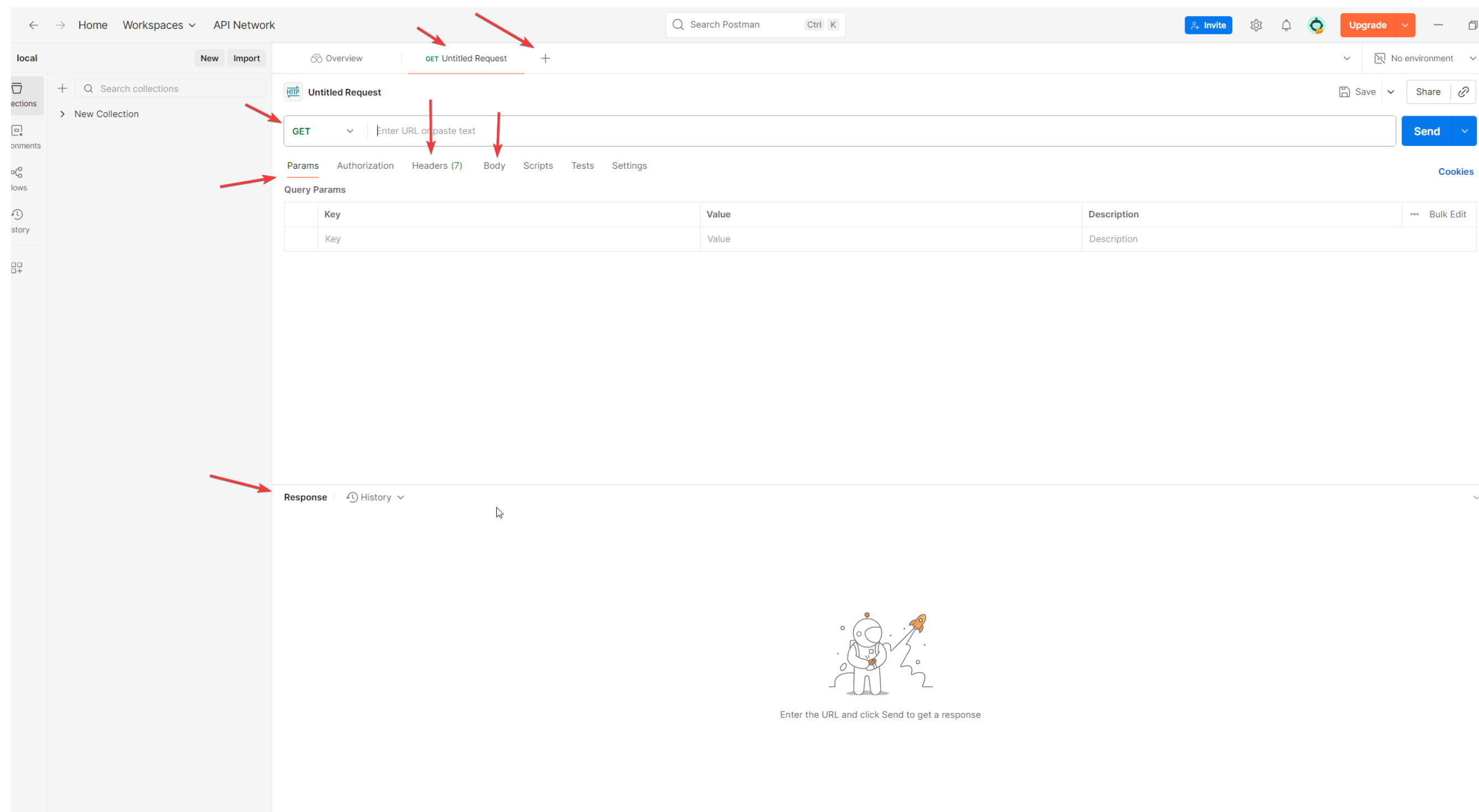
ИНСТРУМЕНТ ДЛЯ ТЕСТИРОВАНИЯ И ОТЛАДКИ REST API



1. Postman — это инструмент для тестирования API без frontend.
2. Позволяет:
 - отправлять HTTP-запросы
 - настраивать заголовки
 - передавать JSON-данные
 - смотреть ответы сервера
3. Фактически Postman имитирует работу клиента.
4. <https://www.postman.com/downloads/>
5. Пошаговая инструкция
 - а. <https://learning.postman.com/docs/getting-started/overview/>

ИНТЕРФЕЙС POSTMAN

ОСНОВНЫЕ ЭЛЕМЕНТЫ РАБОЧЕГО ОКНА



1. Ключевые части интерфейса:

a. Method — HTTP-метод

b. URL — адрес запроса

c. Params — query-параметры

d. Headers — заголовки

e. Body — тело запроса

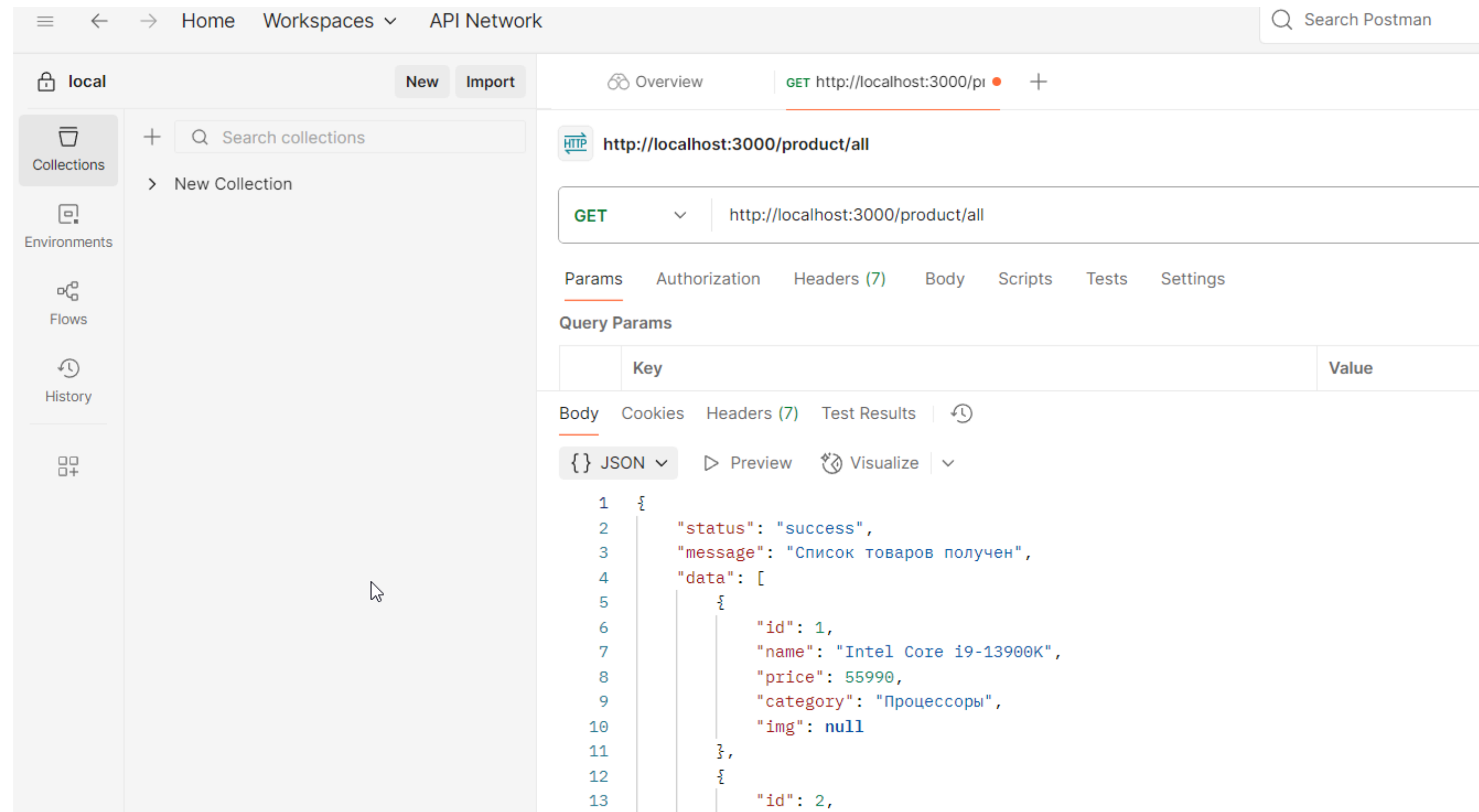
f. Authorization — тип авторизации

g. Send — отправка запроса

h. Response panel — ответ сервера

ОТПРАВКА GET-ЗАПРОСА

ПОЛУЧЕНИЕ ДАННЫХ С СЕРВЕРА



1. URL:

a. `http://localhost:3000/api/users`

2. Шаги:

a. Выбрать **GET**

b. Вставить **URL**

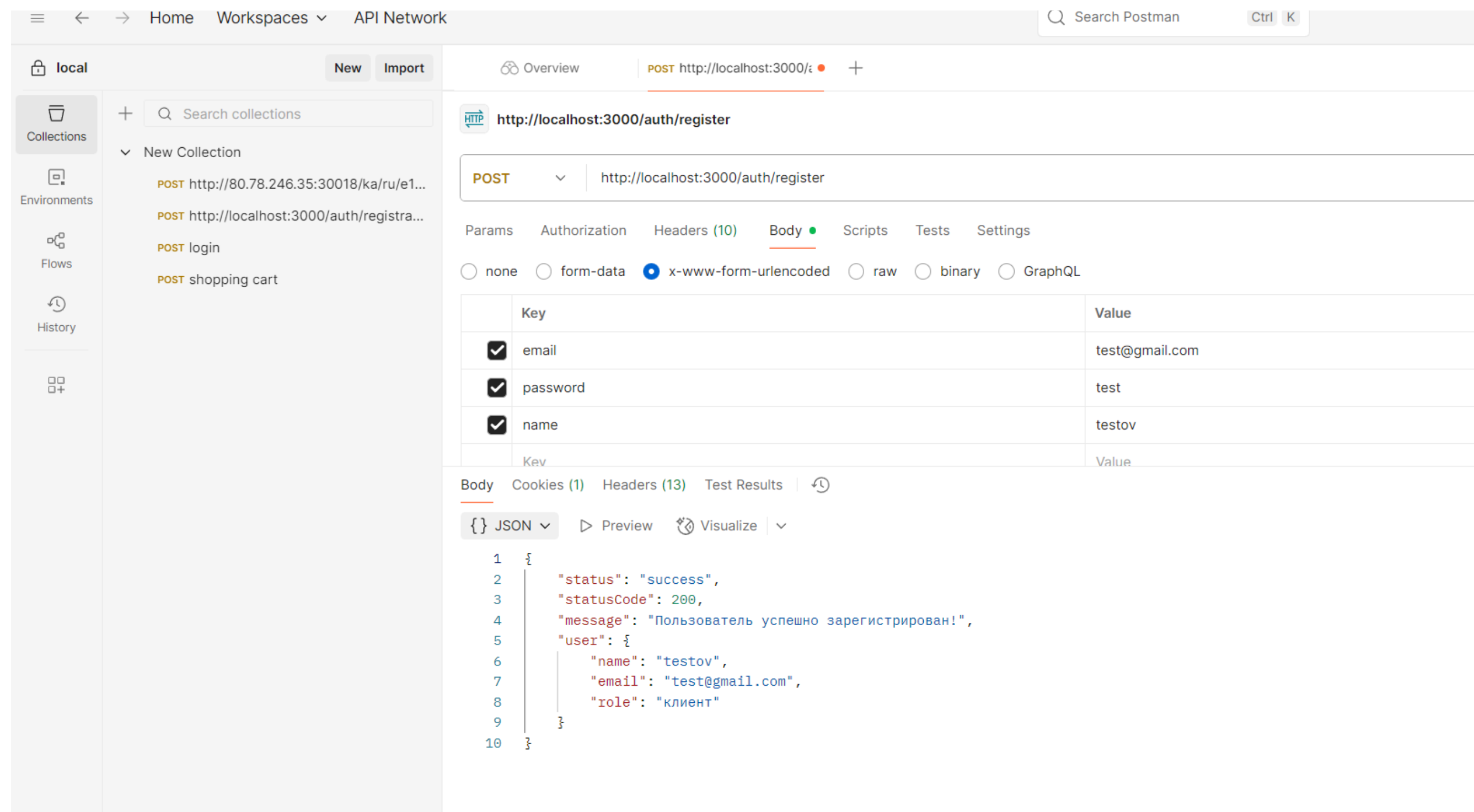
c. Нажать **Send**

d. Проанализировать **Response**

3.

ОТПРАВКА POST-ЗАПРОСА

ПЕРЕДАЧА ДАННЫХ ФОРМЫ



1. URL:

a. `http://localhost:3000/auth/register`

2. Шаги:

a. Выбрать **POST**

b. Вставить **URL**

c. Вкладка **Body** → **x-www-form-urlencoded**

d. **Key Value**

i. email `test@gmail.com`

ii. password `test`

iii. name `test`

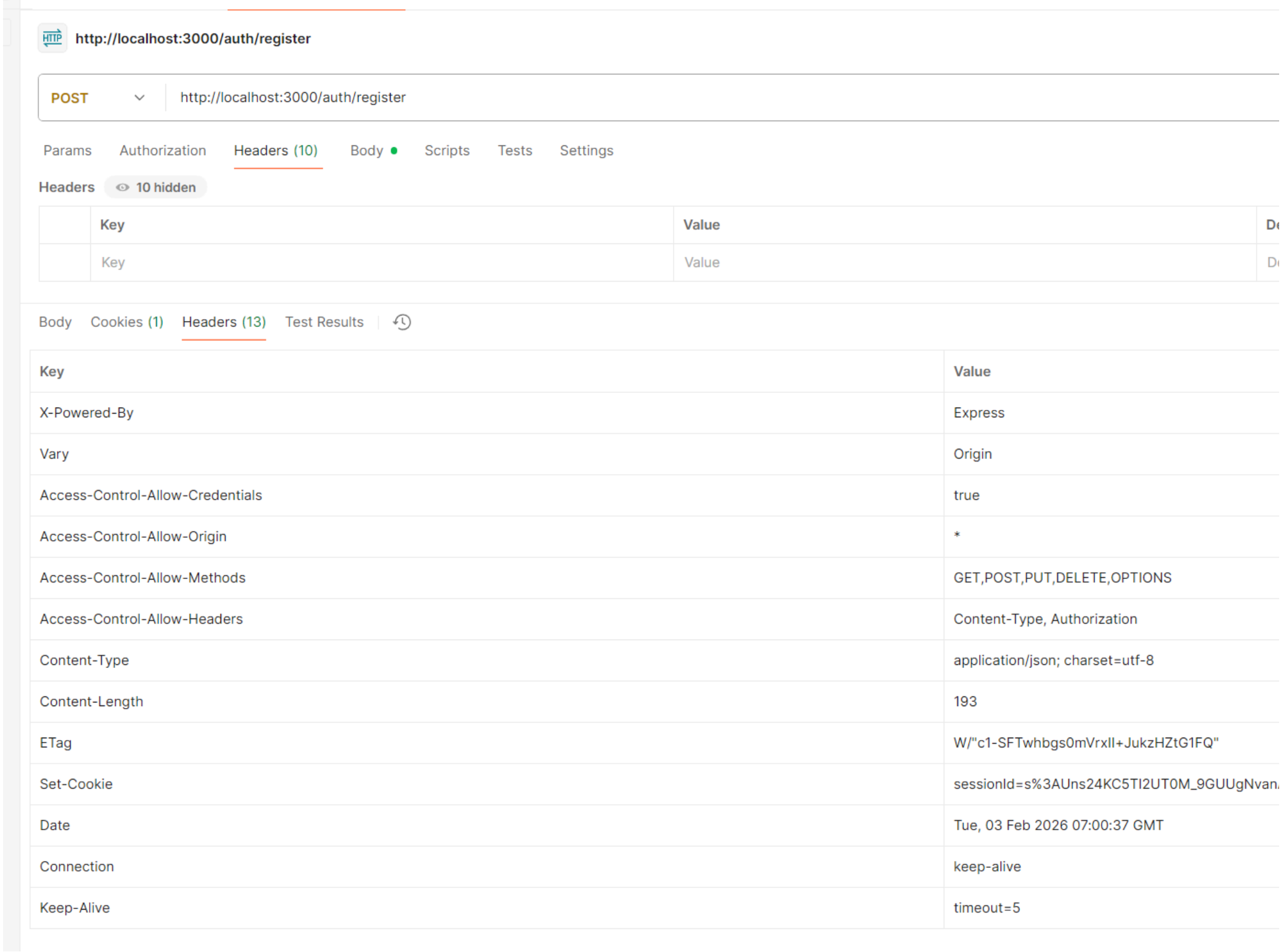
e. Нажать **Send**

f. Проанализировать **Response**

3.

РАБОТА С HEADERS

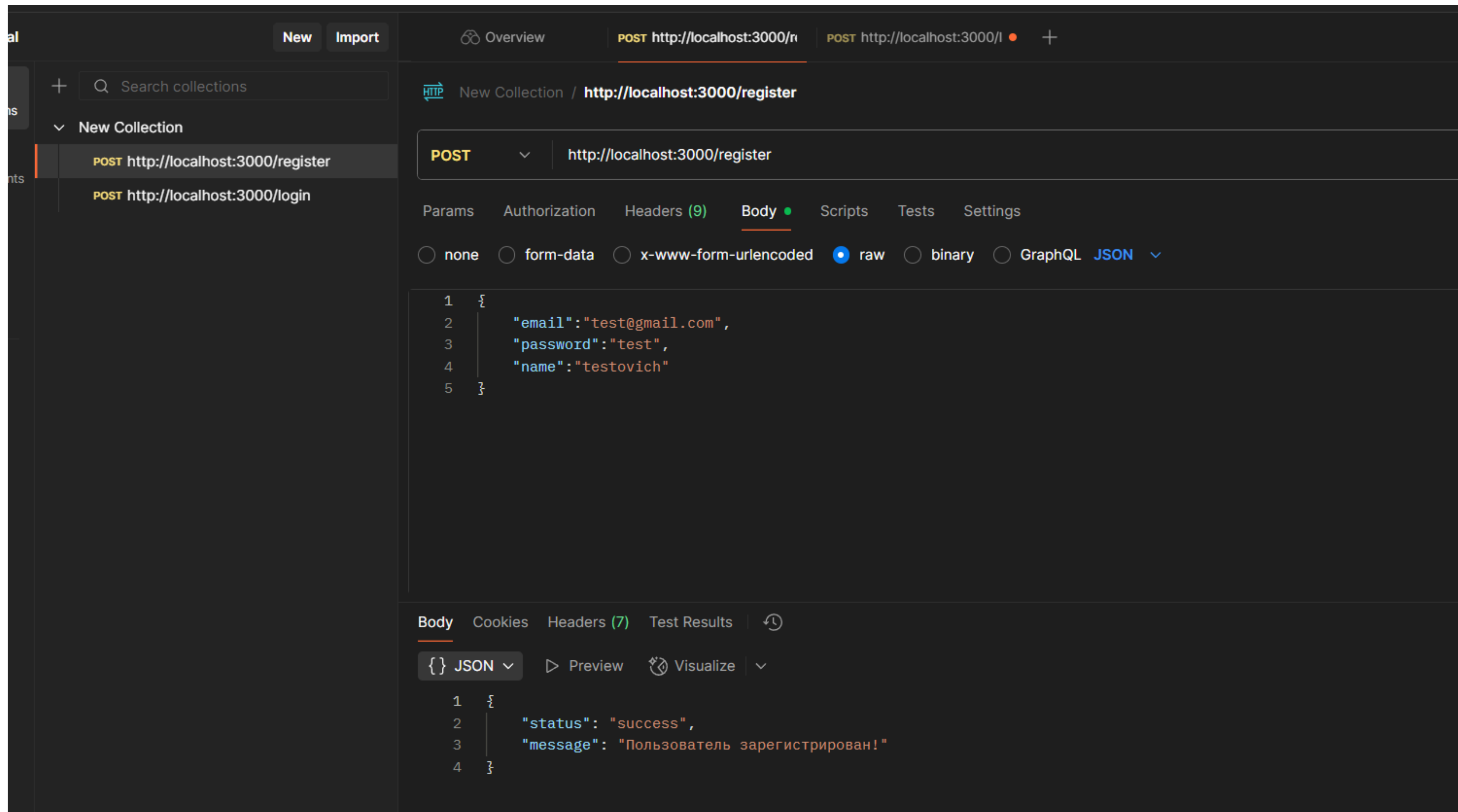
ПЕРЕДАЧА СЛУЖЕБНОЙ ИНФОРМАЦИИ СЕРВЕРУ

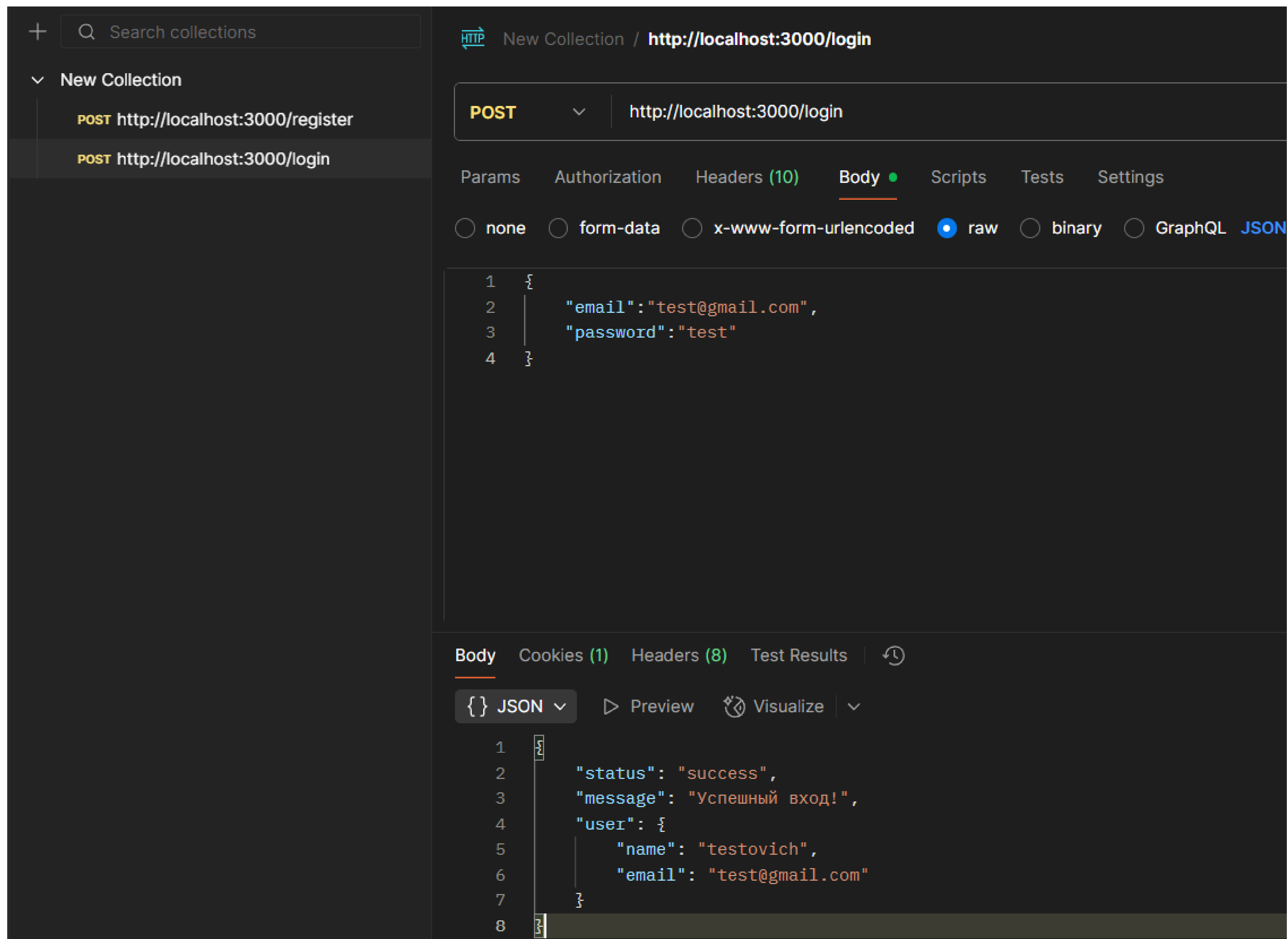


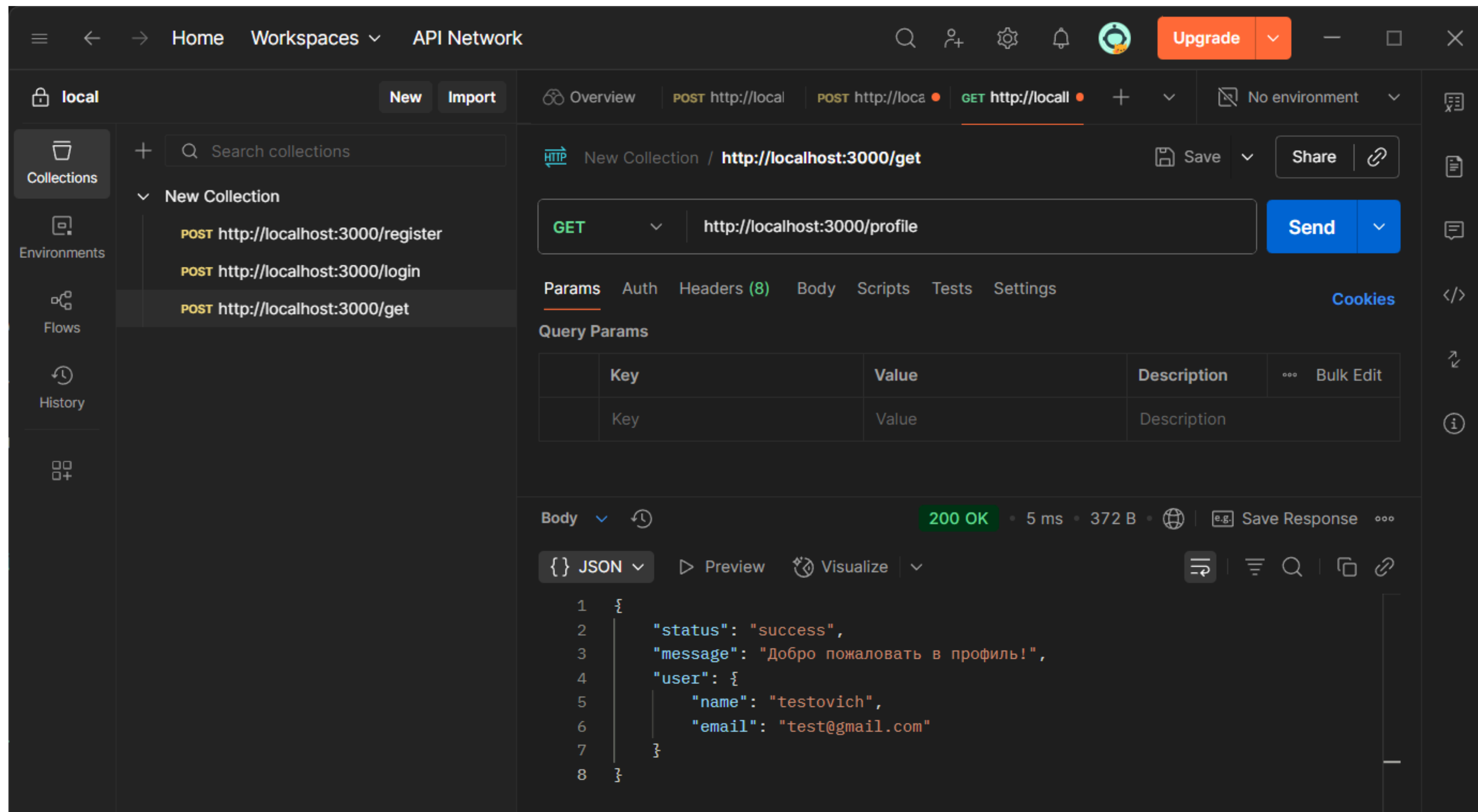
1. Часто используемые заголовки:
- Content-Type - Тип отправляемых данных
 - Authorization - Передача данных авторизации
 - Ассерпт - Какой формат ответа ожидает клиент
 - Cookie - Передача сессионных данных

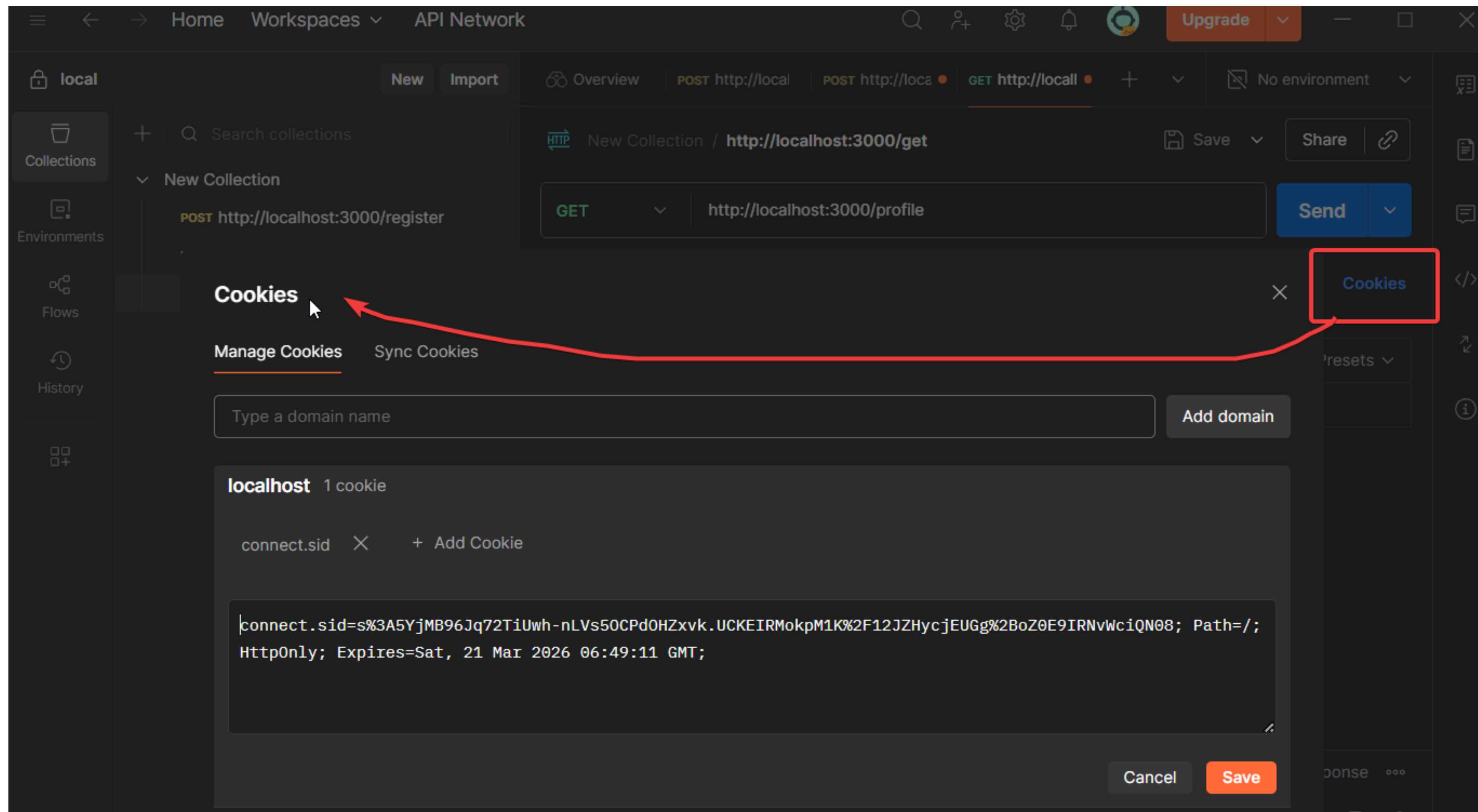
2. Они не содержат основной payload (тело), а управляют обработкой запроса.

- a. **Content-Type** application/json
- b. **Authorization** Bearer token
- c. **Accept** application/json











ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?